

v4.0 — Walking Skeleton: Upload → View

E2E outcome: a signed-in pathologist uploads a 3 GB SVS, and two minutes after ingest completes can pan/zoom it in the browser at full magnification. State machine, audit log, and deploy pipeline exist. Everything later hangs off this.

Est: 1 week.

1. Scope

In: repo scaffold, auth, case CRUD, signed resumable upload, ingest worker (metadata + label strip + checksum), DZI pyramid, tile serving, OpenSeadragon viewer, stage state machine + poll worker, audit log, local + dev deploy.

Out: stain normalization, QC, any ML, review/confirm UI beyond "slide is visible".

2. Backend

2.1 Endpoints

POST /api/v1/cases → {case_id}

POST /api/v1/cases/{id}/slide/upload-url body: {filename, size_bytes, content_type}
→ {upload_url, gcs_uri}

POST /api/v1/cases/{id}/slide/finalize body: {gcs_uri, client_sha256?}
→ queues 'ingest' stage_execution

GET /api/v1/cases/{id} → case + slides + stage statuses

GET /api/v1/cases/{id}/tiles/{layer}/{path:path} → tile bytes (see 2.4)

GET /api/v1/cases/{id}/audit → audit events (paginated)

2.2 Signed resumable upload

- Server:
`blob.create_resumable_upload_session(origin=FRONTEND_ORIGIN, size=size_bytes)` (google-cloud-storage). Returns the session URI — this is the `upload_url`; TTL is inherent to the session.
- Object path: `og-{env}-raw/cases/{case_id}/{uuid4()}{ext}` — **no filename, no patient identifiers** in the path. Original filename stored in DB only.
- Client: upload in 32 MiB chunks with `Content-Range`; on network error, query session status (PUT with `Content-Range: bytes */{size}`) and resume from the returned offset. Use a small wrapper around `fetch` — no upload library needed.
- Validation before issuing URL: extension $\in \{.svs, .ndpi, .mrxs, .tif, .tiff, .dcm\}$, size ≤ 20 GB.

2.3 Ingest handler (`worker/ingest.py`)

1. Download original to worker scratch (Cloud Run: mount tmpfs; size accordingly — 32 GiB memory tier or use a GCE-attached disk if slides > 8 GB are common). Stream-download with `blob.download_to_filename`.
2. `sha256` while downloading (chunked); store `checksum_sha256`.
3. **Label/macro strip**: run `wsi-anon` (open-source, C, supports SVS/NDPI/MRXS in-place associated-image removal). Fallback for formats it can't handle: re-write via `tifftools` removing `label` and `macro` sub-IFDs. Strip **before** any other processing; write stripped copy back over the raw object; set `label_stripped_at`.
4. Open with OpenSlide. Extract: `mpp_x/y` (`openslide.mpp-x`), `objective-power` → `base_mag`, `dimensions`, vendor (`openslide.vendor`) → `scanner`, `format`. If MPP missing (some TIFFs): mark slide `status=needs_mpp`, surface a manual MPP entry field in UI — **never guess MPP**; every downstream μm computation depends on it.
5. Pyramid: `pyvips.Image.new_from_file(path, access="sequential")` → `dzsave` with `tile_size=256`, `overlap=0`, `suffix=".jpg[Q=90]"`, output to scratch dir, then parallel-upload the DZI tree to `og-{env}-pyramids/{slide_id}/orig/` (`transfer_manager.upload_many_from_filenames`, 32 workers).
6. Write `slides` row fields; emit `stage_output`; ingest stage → `done` (no review gate for v4.0).

Benchmarks to verify (acceptance): 100k×80k SVS → pyramid in ≤ 8 min on 8 vCPU; peak scratch usage < 2.5× original size.

2.4 Tile serving

v4.0: proxy through FastAPI — `GET /tiles/{layer}/{z}/{x}_{y}.jpg` streams from GCS with auth check (case membership) + `Cache-Control: private, max-age=86400`.

2.5 Auth

Firebase Auth, Google provider. FastAPI dependency verifies the ID token (`firebase_admin.auth.verify_id_token`), maps to `users` table (`id`, `email`, `role`, `created_at`), `role` $\in$ {pathologist, technician, viewer}. All case queries filtered by creator/institution (single-tenant for now: any authenticated pathologist sees all cases — multi-tenant scoping is v5).

3. Frontend

- `/cases` list + "New case" → upload dropzone (shows chunk progress, survives refresh via localStorage of session URI).
- `/cases/{id}` — the **case workspace**: left rail = stage list with statuses (populated by polling `GET /cases/{id}` every 3 s; WS later if needed); center = viewer.
- Viewer component: OpenSeadragon with custom `tileSource`:

```
{ width, height, tileSize: 256, overlap: 0,
```

```
  minLevel: 8, // skip levels coarser than the DZI has
```

```
  getTileUrl: (z, x, y) => `/api/v1/cases/${id}/tiles/orig/${z}/${x}_${y}.jpg` }
```

Wrapper exposes an imperative API used by all later stages: `addOverlayLayer(id, svgEl)`, `umToViewport(x_um, y_um)`, `onViewportClick(cb)`. Get this interface right now — v4.2–v4.4 build on it.

- Scale bar: compute from `mpp_x` × current zoom (OSD `ScalebarPlugin` or 30 lines custom).

4. Infra / ops

- `docker-compose.yml`: postgres:16 + fake-gcs-server (`fsouza/fake-gcs-server`) with `-scheme http`; env var `STORAGE_EMULATOR_HOST` switches the GCS client.
- Dockerfiles: `backend` (installs `libopenslide1`, `libvips`, `wsr-anon` binary), `frontend` (node:22-slim, standalone Next build).
- Cloud Run services: `og-api` (2 vCPU/4 GiB), `og-worker` (8 vCPU/32 GiB, min-instances 0, request-less: run as always-on polling service with min-instances 1), `og-web`. Cloud SQL via connector.
- `make dev` / `make deploy-dev` (gcloud CLI scripts; skip Terraform for now).

5. Tests

- `pipeline/coords.py`: round-trip um $\leftrightarrow$ px property tests at 3 MPP values.
- Ingest handler against a small public SVS (CMU-1-Small-Region.svs, 2 MB, in OpenSlide test data) in CI: asserts metadata, checksum, pyramid tile count, label absence (`openslide.associated_images` empty post-strip).
- API auth: unauthenticated tile request $\rightarrow$ 401.

6. Acceptance checklist

- ☐ 3 GB SVS uploads on throttled connection (Chrome dev-tools 20 Mbps), survives one forced disconnect, resumes.
- ☐ TCGA SVS + Hamamatsu NDPI both ingest and render; scale bar correct (verify against slide ruler if present).
- ☐ Label/macro images gone from stored object (checked via OpenSlide associated images).
- ☐ `audit_events` shows case_created $\rightarrow$ slide_uploaded $\rightarrow$ stage_started $\rightarrow$ stage_output for the run.
- ☐ Deployed on dev; a second machine can log in and view the slide.